

# LOCIM — PayPal AI Hackathon Build Plan

## 1. Project Overview

---

**Project:** LOCIM

**Positioning:** An AI-native freelance marketplace that turns a client's idea into a structured, funded, and trackable freelance project.

**Core concept:**

| From idea → freelancer → milestone → payment, with AI in the middle.

LOCIM combines AI project planning, freelancer matching, milestone management, scope-change detection, deliverable analysis, and PayPal payments.

The project will use the PayPal Sandbox for the hackathon.

## 2. Hackathon Goal

---

Build a working prototype that demonstrates a complete AI-assisted freelance transaction:

1. Client describes a project in natural language.
2. AI converts the request into a structured project.
3. AI recommends milestones and required skills.
4. LOCIM matches suitable freelancers.
5. Client selects a freelancer.
6. Project milestones are created.
7. Client funds a milestone through PayPal Sandbox.
8. Freelancer submits work/evidence.
9. AI analyzes the submission against milestone requirements.
10. Client approves the milestone.
11. LOCIM records the PayPal transaction.
12. AI detects potential scope changes and can create additional milestones.

PayPal must be central to the transaction lifecycle, not merely a checkout button.

## 3. Technology Stack

---

### Frontend

- Next.js
- TypeScript
- Tailwind CSS
- shadcn/ui where useful

### Backend

- Python
- FastAPI
- SQLAlchemy
- Pydantic
- Alembic

### Database

- PostgreSQL

### AI

- LLM API with structured JSON outputs
- Specialized AI services instead of one monolithic AI endpoint

### Payments

- PayPal Developer Platform

- PayPal Sandbox
- Orders/checkout flow appropriate to the selected PayPal integration

## Infrastructure

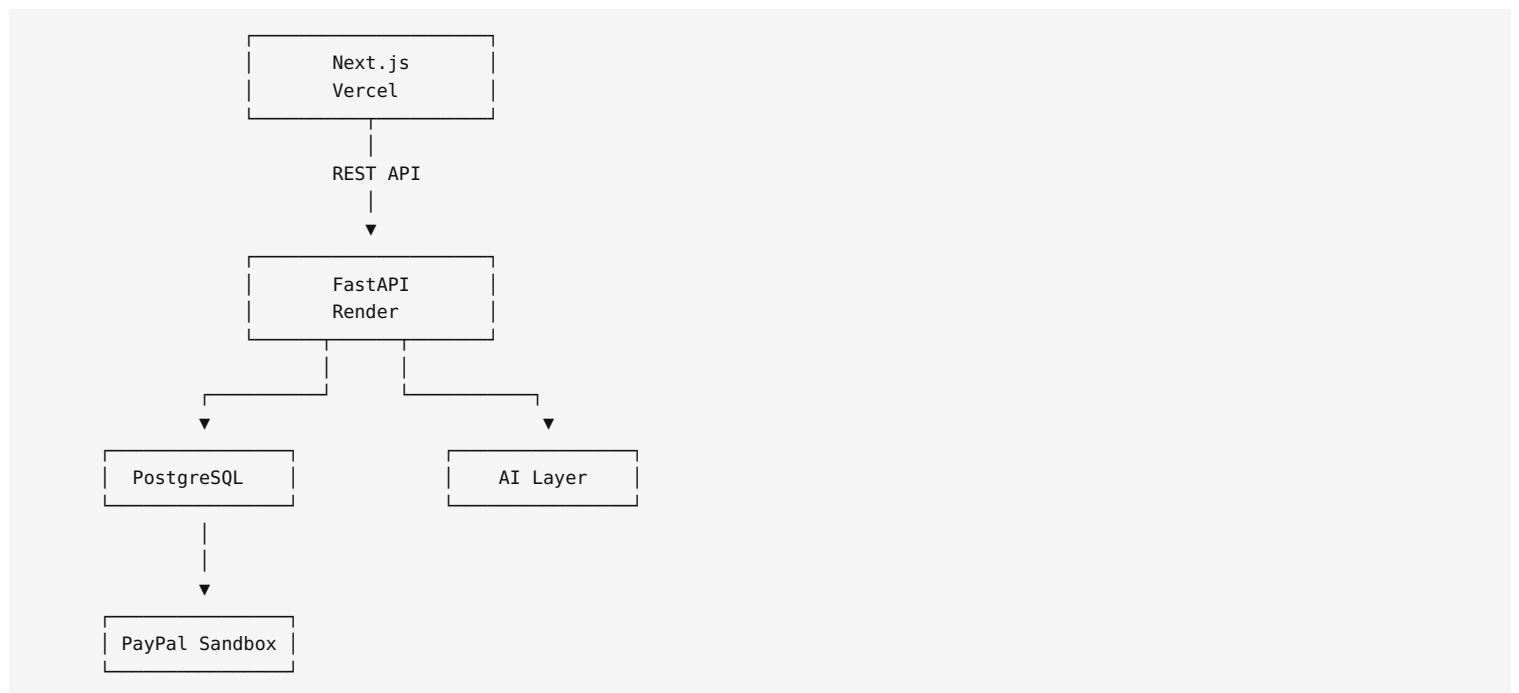
- Docker / Docker Compose for local development
- Vercel for frontend deployment
- Render for FastAPI backend
- PostgreSQL hosted alongside the backend or through the selected PostgreSQL provider

## Repository

- Public GitHub repository
- Open-source license visible in repository
- README with complete setup instructions

## 4. Product Architecture

---



## 5. Core User Roles

---

### Client

- Create projects
- Describe requirements using natural language
- Review AI-generated project plans
- View freelancer recommendations
- Select a freelancer
- Review milestones
- Fund milestones through PayPal
- Review submissions
- Approve/reject milestones
- Create scope-change requests

### Freelancer

- Create profile
- Add skills and portfolio
- View recommended projects
- Accept projects
- View milestones
- Submit work and evidence
- Respond to revision requests
- Track payment status

## 6. Core AI Capabilities

### AI Project Architect

Input:

I need a restaurant website for around \$500 within two weeks.

Output:

- Project title
- Description
- Required skills
- Budget
- Deadline
- Milestones
- Milestone descriptions
- Suggested amounts

The backend should validate AI output using Pydantic schemas.

### AI Freelancer Matching

Analyze:

- Required skills
- Project type
- Portfolio relevance
- Previous experience
- Budget compatibility
- Availability

Display an explainable compatibility result rather than relying only on a score.

Example:

92% compatibility

Skill match

95%

Portfolio match

90%

Budget fit

92%

Availability

90%

Why:

✓ Built similar projects

✓ Required React/Next.js experience

✓ Available within deadline

✓ Previous projects fit the budget

### AI Scope Change Detection

Compare a new client/freelancer request with the original project scope.

Example:

Original:

- Restaurant landing page
- Menu
- Contact form

New request:

Can we add online reservations?

LOCIM:

Potential scope change detected.

The requested reservation functionality was not included in the original project scope.

Suggested new milestone:  
Reservation System  
Suggested amount: \$100

The AI suggests; the user decides.

AI Submission Analysis

Compare submitted evidence with milestone requirements.

Input:

- Milestone requirements
- Freelancer description
- GitHub URL
- Demo URL
- Optional screenshots

Output:

4/5 requirements appear satisfied.

Detected:  
✓ Hero section  
✓ Navigation  
✓ Menu section  
✓ Contact section

Potentially missing:  
△ Mobile navigation

Status:  
Needs client review

The AI should not automatically release or deny payments.

7. Database Design

users

id  
name  
email  
password\_hash / auth\_identifier  
role  
created\_at  
updated\_at

Roles:

- CLIENT
- FREELANCER

freelancer\_profiles

id  
user\_id  
headline  
bio  
skills  
hourly\_rate  
availability  
portfolio\_url  
created\_at  
updated\_at

projects

id  
client\_id  
title  
description

budget  
currency  
deadline  
status  
ai\_generated\_requirements  
created\_at  
updated\_at

proposals

id  
project\_id  
freelancer\_id  
cover\_letter  
proposed\_price  
status  
ai\_match\_score  
ai\_match\_explanation  
created\_at

milestones

id  
project\_id  
title  
description  
amount  
currency  
due\_date  
sequence  
status  
created\_at  
updated\_at

submissions

id  
milestone\_id  
freelancer\_id  
description  
repository\_url  
demo\_url  
screenshots  
ai\_analysis  
status  
created\_at

payments

id  
milestone\_id  
paypal\_order\_id  
amount  
currency  
status  
paypal\_response  
created\_at  
updated\_at

scope\_change\_requests

id  
project\_id  
requested\_by  
description  
ai\_analysis  
suggested\_milestone  
suggested\_amount  
status  
created\_at

8. Main API Modules

```
/api/auth
/api/users
/api/projects
/api/projects/{id}
/api/projects/{id}/proposals
/api/freelancers
/api/matching
/api/milestones
/api/milestones/{id}
/api/submissions
/api/payments
/api/payments/paypal/create-order
/api/payments/paypal/capture
/api/ai/project
/api/ai/match
/api/ai/submission
/api/ai/scope-change
```

The exact endpoint structure may be refined during implementation.

## 9. PayPal Payment Flow

```
Client
↓
Select milestone
↓
LOCIM backend validates milestone
↓
FastAPI creates PayPal Sandbox order
↓
Frontend opens PayPal checkout
↓
Sandbox approval
↓
FastAPI captures order
↓
LOCIM verifies response
↓
Payment record created
↓
Milestone marked as funded/paid according to workflow
```

Important:

- Never trust payment status supplied by the browser.
- Store PayPal order IDs.
- Keep PayPal credentials server-side.
- Use sandbox accounts during development/demo.
- Do not describe sandbox transactions as real money.

## 10. Frontend Pages

### Public

- Landing page
- Login
- Register
- How LOCIM works

### Client

- Client dashboard
- AI project creation
- Project details
- Freelancer recommendations
- Freelancer profile
- Milestone dashboard
- PayPal payment
- Submission review
- Scope change review

- Payment history

## Freelancer

- Freelancer dashboard
- Profile
- Recommended projects
- Project details
- Milestone details
- Submission form
- Payment history

## Demo

A controlled demo environment may be added to make the 3-minute presentation reliable.

# 11. Folder Structure

## Frontend

```
frontend/
├── app/
│   ├── dashboard/
│   ├── projects/
│   ├── freelancers/
│   ├── payments/
│   ├── ai/
│   └── auth/
├── components/
├── lib/
│   ├── api.ts
│   └── paypal.ts
├── hooks/
├── types/
└── Dockerfile
```

## Backend

```
backend/
├── app/
│   ├── main.py
│   ├── api/
│   ├── models/
│   ├── schemas/
│   ├── services/
│   │   ├── ai_service.py
│   │   ├── matching_service.py
│   │   ├── paypal_service.py
│   │   └── milestone_service.py
│   ├── db/
│   └── core/
├── tests/
├── alembic/
├── Dockerfile
└── requirements.txt
```

# 12. Docker

Local services:

```
frontend
backend
postgres
```

Use Docker Compose for local development.

The application should also be runnable without Docker if practical, but Docker is the primary reproducible development environment.

# 13. Deployment

## Frontend

Vercel:

- Next.js application
- Production environment variables
- Backend API URL

## Backend

Render:

- FastAPI service
- Docker deployment
- AI API key
- PayPal credentials
- Database connection string
- CORS configuration

## Database

PostgreSQL:

- Production database
- Automated migrations
- Separate development and production credentials where possible

## 14. Security Basics

---

- Never expose PayPal client secrets to the browser.
- Never expose AI API keys.
- Validate all AI-generated structured data.
- Validate all payment amounts against database records.
- Do not accept arbitrary payment amounts from the client.
- Verify PayPal order/capture results on the server.
- Use environment variables for secrets.
- Add authentication before production deployment.
- Add authorization checks for client/freelancer resources.
- Sanitize URLs and user-submitted content.
- Avoid storing unnecessary payment information.

## 15. MVP Scope Lock

---

### Must Have

- Client and freelancer accounts
- AI project creation
- AI milestone generation
- Freelancer profiles
- AI freelancer matching
- Project management
- Milestones
- PayPal Sandbox payment
- Freelancer submission
- AI submission analysis
- Scope-change detection
- Responsive UI
- Public GitHub repository
- Hosted demo
- README
- Demo video

### Should Have

- Payment history

- Notifications
- Project activity timeline
- Better freelancer search
- AI-generated project summaries
- Demo seed data

### Nice to Have

- Real-time notifications
- Messaging
- Advanced analytics
- Reputation system
- AI contract generation
- Dispute workflow
- Multiple currencies
- Additional payment providers

Do not allow Nice-to-Have features to delay the core demo.

## 16. 41-Day Development Roadmap

---

### Days 1-3 — Project Setup

- Create repository
- Define architecture
- Initialize Next.js
- Initialize FastAPI
- Configure PostgreSQL
- Configure Docker Compose
- Establish frontend/backend communication
- Set environment variable conventions

### Days 4-7 — Authentication + Users

- Client registration/login
- Freelancer registration/login
- User roles
- Protected routes
- Basic dashboards
- Freelancer profile

### Days 8-12 — AI Project Architect

- AI service abstraction
- Structured output schemas
- Project generation prompt
- Milestone generation
- Requirements extraction
- Save AI-generated project data
- Build AI project creation UI

### Days 13-16 — Freelancer Marketplace

- Freelancer profiles
- Project listing
- Project detail page
- Matching service
- AI matching explanation
- Proposal flow
- Client freelancer selection

### Days 17-20 — Project + Milestones

- Project lifecycle
- Milestone CRUD
- Milestone statuses
- Freelancer project view

- Client project view
- Submission model
- Activity timeline

## **Days 21-24 — PayPal Integration**

- Create PayPal developer/sandbox setup
- Configure credentials
- Implement create-order endpoint
- Implement checkout UI
- Implement capture endpoint
- Store PayPal transaction/order ID
- Payment status handling
- End-to-end sandbox testing

## **Days 25-27 — Submission AI**

- Freelancer submission UI
- GitHub/demo evidence
- AI milestone analysis
- Requirement comparison
- Client review interface

## **Days 28-30 — Scope Change Intelligence**

- Scope change request UI
- AI comparison against original scope
- Suggested additional milestone
- Client approval
- PayPal payment flow for new milestone

## **Days 31-34 — UX + Visual Polish**

- Landing page
- Dashboard polish
- Loading states
- Error states
- Empty states
- Responsive design
- Consistent design system
- Demo-friendly data

## **Days 35-37 — Deployment**

- Deploy PostgreSQL
- Deploy FastAPI to Render
- Deploy Next.js to Vercel
- Configure CORS
- Configure environment variables
- Test production PayPal Sandbox
- Fix deployment issues

## **Days 38-39 — Reliability**

- Full end-to-end test
- Authentication test
- Payment test
- AI failure handling
- Database migration test
- Mobile/responsive test
- Clean GitHub repository
- README

## **Day 40 — Demo Production**

Create a controlled 3-minute demo:

1. Client describes project

2. AI generates project
3. AI recommends freelancer
4. Client selects freelancer
5. Milestones appear
6. Client funds milestone through PayPal Sandbox
7. Freelancer submits work
8. AI analyzes submission
9. Client requests scope change
10. LOCIM creates additional milestone
11. PayPal payment flow
12. Final project/payment dashboard

Record multiple takes and choose the clearest one.

Day 41 — Submission

- Final GitHub push
- Verify repository is public
- Verify license
- Verify README
- Verify hosted URL
- Verify demo video is public
- Complete Devpost project description
- Document all tools
- Explain PayPal integration
- Explain AI functionality
- Final QA
- Submit

17. Demo Story

Use one fictional project throughout the entire demo.

Example:

"I need a restaurant website for \$500 and I need it within two weeks."

LOCIM generates:

|                      |       |
|----------------------|-------|
| Restaurant Website   |       |
| Budget: \$500        |       |
| Deadline: 14 days    |       |
| Homepage             | \$125 |
| Menu                 | \$125 |
| Contact/Backend      | \$150 |
| Testing & Deployment | \$100 |

LOCIM recommends a freelancer based on:

- skills
- portfolio
- project experience
- budget
- availability

Client selects the freelancer.

The first milestone is funded through PayPal Sandbox.

Freelancer submits a GitHub repository and demo URL.

LOCIM analyzes the milestone:

|                                       |
|---------------------------------------|
| 4/5 requirements detected             |
| Potential issue:                      |
| Mobile navigation appears incomplete. |

The client then asks:

"Can we add online reservations?"

LOCIM detects a potential scope change.

It proposes:

|                    |
|--------------------|
| Reservation System |
| \$100              |

The client approves the additional milestone.

The demo ends with the updated project and payment state.

## 18. Hackathon Positioning

Avoid presenting LOCIM as:

"An AI version of Upwork."

Instead present it as:

**LOCIM is an AI-native freelance transaction layer. It transforms natural-language project intent into structured work, matches clients with freelancers, manages milestone-based collaboration, and connects those milestones to PayPal payments.**

The differentiating idea is:

The client does not need to understand the mechanics of a freelance marketplace. They describe what they want, and LOCIM turns that intent into a structured project and transaction.

## 19. What Makes PayPal Central

PayPal should appear in three important moments:

### 1. Fund a milestone

AI creates the milestone; PayPal executes the payment.

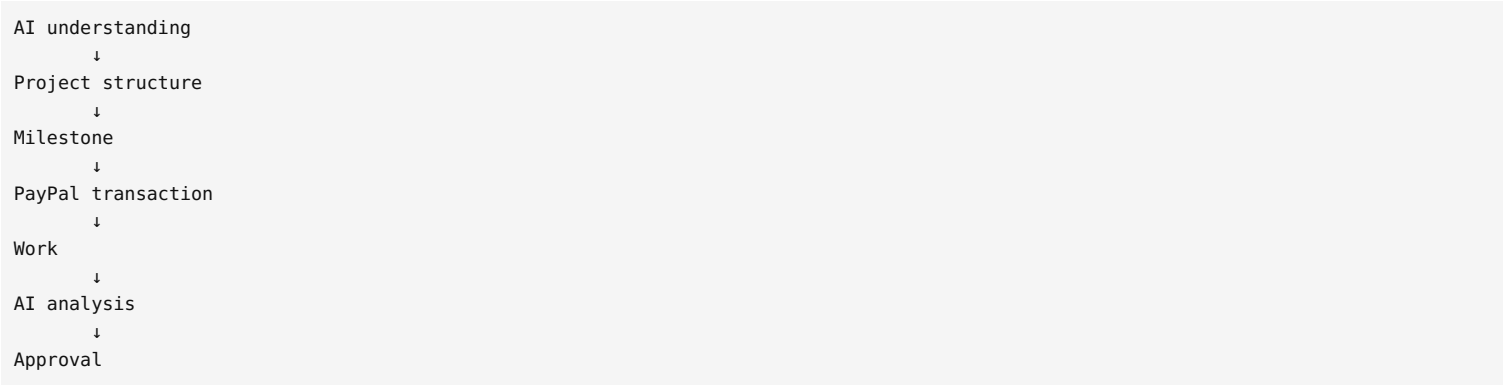
### 2. Scope expansion

AI identifies a potential scope change; LOCIM creates a new milestone; PayPal handles the additional payment.

### 3. Payment history

LOCIM connects project milestones to PayPal transaction records.

The hackathon judges should be able to clearly see:



## 20. Success Criteria

The project is ready when a judge can:

- Open the hosted application.
- Understand what LOCIM does quickly.

- Create a project using natural language.
- See AI-generated requirements and milestones.
- See AI freelancer matching.
- Select a freelancer.
- Complete a PayPal Sandbox payment.
- See the payment attached to a milestone.
- View a freelancer submission.
- See AI analysis.
- Trigger a scope-change workflow.
- Understand why PayPal and AI are both essential.

## 21. Final Product Statement

---

### LOCIM

▮ **From idea to work to payment — with AI in the middle.**

LOCIM doesn't simply help people find freelancers.

It helps turn an idea into a structured project, connects that project to the right person, tracks the work, identifies changes in scope, and turns approved milestones into PayPal-powered transactions.

The goal is not to rebuild an entire freelance marketplace.

The goal is to demonstrate a new AI-native way for people to commission work and move money.